omnisient

Omnisient Jupyter Notebook User Manual

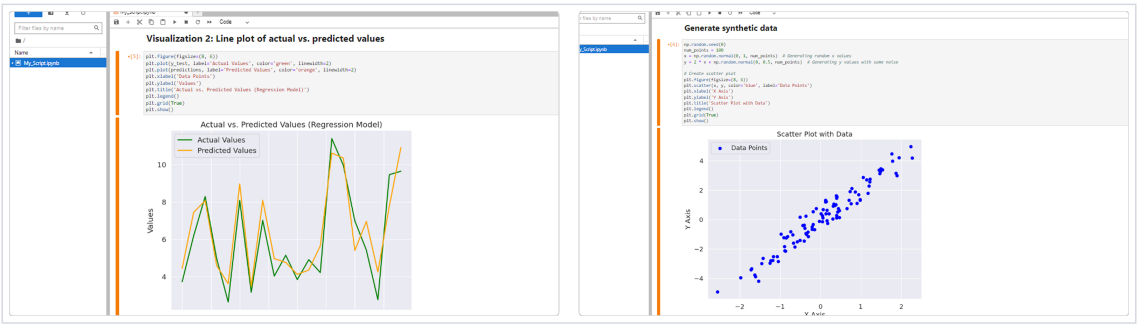
Aspects of the Omnisient™ Platform is protected by patent.
US Patent No. 11,748,515 and patent pending GB2315353.9

Jupyter Notebook

Python, a widely used programming language for data scientists, offers a wide selection of libraries that you can install with the pip command, simplifying complex calculations and data analysis tasks. With Python integrated into Omnisient, data scientists can seamlessly work on anonymized data without the hassle of moving data around. It also facilitates the automation of tasks and processes, streamlining workflow for enhanced productivity and efficiency.

Jupyter Notebook is an open-source web application that's become one of the go-to tools for data scientists, analysts, and even educators in recent years. It provides an environment where you can write live code, equations, visualizations, and narrative text - all in a single document, making it a versatile choice for a variety of data projects.

What sets Jupyter Notebook apart is its ability to combine code execution with data visualization and rich text, facilitating a more interactive and insightful data exploration process. You can jot down your thought process while simultaneously running code snippets, making it an excellent tool for both the development and sharing of data analyses.



Getting Started

Two things must be in place before you can access Jupyter Notebook in Omnisient:

- An active Jupyter Notebook subscription - your organization must have Jupyter Notebook enabled as part of your plan.
- A compatible project type - the project you are working in must have analytical tools enabled.

> If Jupyter Notebook is not showing in your project, it is likely one of these two reasons.

You can access Jupyter Notebook from two places:

Access point	How
From a project	Open your project and select Open With > Jupyter Notebook.
Own Data Preparation	Available via the Datasets and the project sections in the side navigation.

Each user has their own isolated Jupyter environment, you won't see other users' notebooks or

© 2014-2026. Omnisient (Pty) Ltd. All rights reserved. - www.omnisient.com - Legal Policies

files, and your workspace can be configured to suit your needs.

> Having trouble loading Jupyter? Refresh the page and try again. Check your project status - Jupyter will not load if the project is in a Processing or Failed state. Log out and log back in to refresh your session. Check with your IT team that all required URLs and IP ranges are whitelisted - see the Firewall Access Required article for details.

Workspace size options

Three workspace sizes are available, allowing you to match compute resources to the demands of your work:

Workspace Type	CPU	Memory (RAM)
Standard	2	8GB
Medium	3	25GB
Large	4	50GB
Extra Large	12	108GB

Resources in Omnisient's AKS environment are allocated dynamically and managed by Kubernetes, which ensures efficient utilization of available resources while maintaining performance and scalability.

1. Resource limits and requests - each project is allocated resources based on predefined minimum and maximum limits. The minimum (resource request) ensures each Jupyter instance has enough resources to start and run. The maximum (resource limit) prevents any single instance from consuming excessive resources, ensuring fair distribution across all users.
2. Pod initialization - when a user opens a Jupyter instance, Kubernetes uses the defined resource requests and limits to create a new instance, known as a pod. A pod is a virtual environment that encapsulates the Jupyter application and its dependencies.
3. Scaling mechanism - the AKS setup includes a single server and two scale set instances to handle varying workloads. When resource usage reaches a threshold, Kubernetes automatically provisions additional instances. When demand decreases, it scales down and frees up resources.
4. Virtual server fit - each pod fits within the size constraints of the virtual server in use. Kubernetes continuously monitors resource usage and adjusts allocation to optimize performance.
5. Resource monitoring and management - Kubernetes provides built-in mechanisms for monitoring usage and managing pod lifecycle, ensuring each Jupyter instance operates within defined resource limits without impacting overall system performance.

File browser

When you launch Jupyter in a project, the left panel opens by default to the File Browser. This displays your persisted storage - files, notebooks, and code saved here are maintained

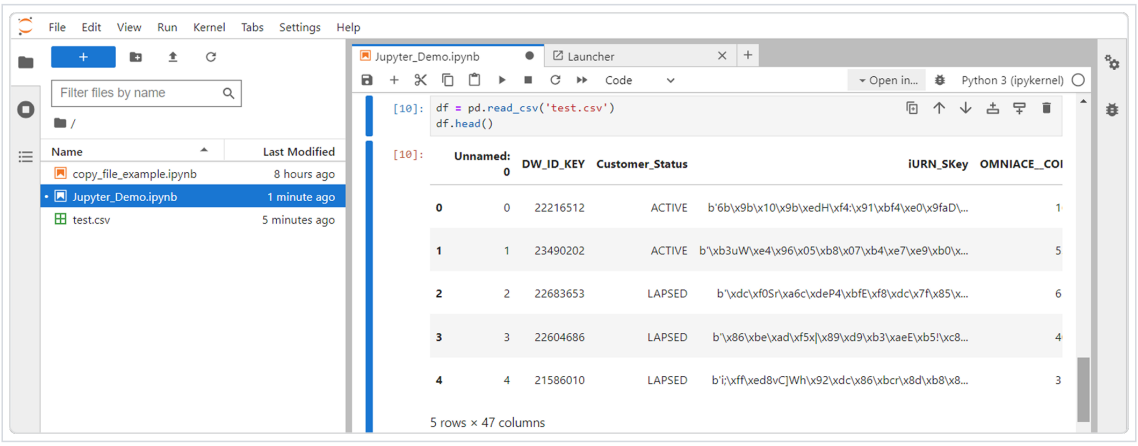
© 2014-2026. Omnisient (Pty) Ltd. All rights reserved. - www.omnisient.com - Legal Policies

between Jupyter sessions, so your work is preserved each time you return to the project.

From the file browser you can:

- Create a new launcher - add a new .ipynb notebook, text file, .py file, and more
- Upload a file from your local computer to your project
- Create a folder within your file browser space

You can also write files out from your notebook directly to the file browser. Files are written to the project rather than to your local disk. For example, a dataframe can be written out as a .csv using standard Pandas functionality. In a later session, you can read the file back into memory:



Any file that can be created using Python scripting can be written and stored in persisted storage.

Right-click context menu

Right-clicking a file in the file browser reveals additional options. Most of these are standard Jupyter actions - Open, Rename, Delete, Cut, Copy, and so on. One option is specific to Omnisient:

Promote to Library: submits a notebook for review and inclusion in your organization's Company Notebook Library. See Company Notebook Library below.

Working with Your Data in Jupyter

OmniHub reads data from the data warehouse into a Jupyter notebook, either from your own Data Preparation area or a project's shared tables. Queries are written as SQL strings and submitted through the omni_hub magic command; the result is a ResultSet object you convert to a pandas DataFrame with .DataFrame().

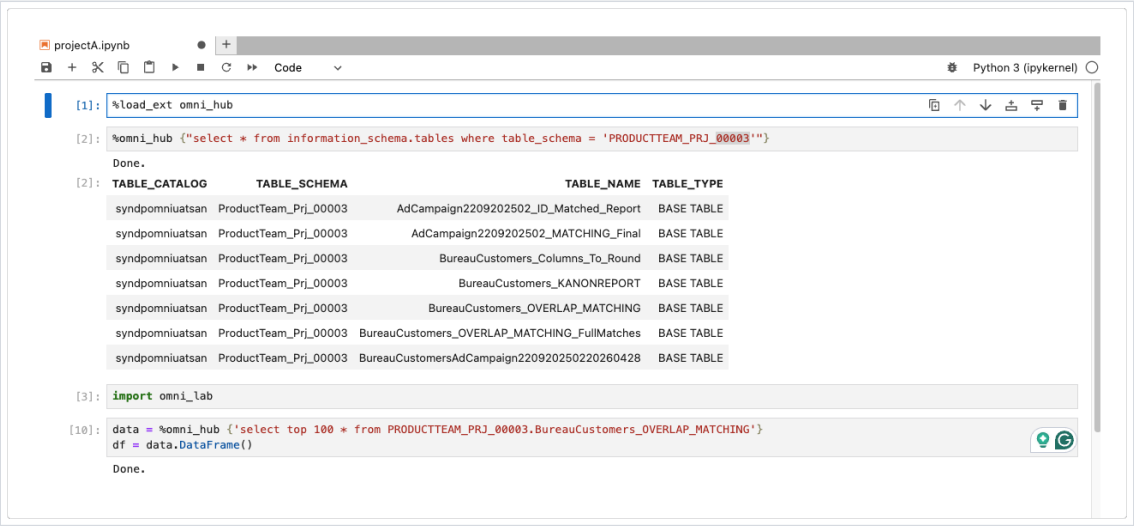
Viewing Available Tables

Before querying, load the extension and list the tables available to you:

1. Load the extension: %load_ext omni_hub
2. Return every table in a project: %omni_hub {"select * from information_schema.tables where table_schema = '<PROJECT_NAME>'"}
3. Replace <PROJECT_NAME> with the working project name, for example

© 2014-2026. Omnisient (Pty) Ltd. All rights reserved. - www.omnisient.com - Legal Policies

OMNISIENT_PRJ_00114. This returns the names, schemas, and types of tables available.



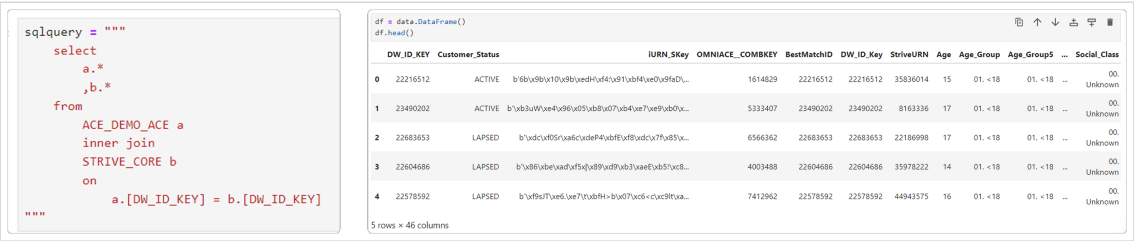
Querying Data

Define the query as a SQL string, submit it through omni_hub, then convert the result to a DataFrame:

```
%load_ext omni_hub
data = %omni_hub {'select top 100 * from [project_code].[table_name]'}
df = data.DataFrame()
```

- Replace [project_code] with the project code, for example ClientName_Prj_00001. For Own Data Preparation, use ClientName_a.
- Replace [table_name] with the table to query.
- top 100 limits the extract to the first 100 rows. Remove it to pull the full table.

Queries use Microsoft SQL Server syntax, and can join a table against a partner's table within the same query - join on DW_ID_KEY or BestMatchID.



Example SQL query joining a project table with a partner table on DW_ID_KEY, run through the omni_hub magic command.

To identify a username for support queries or table-creation permission requests:

```
%omni_hub {'select current_user'}
```

© 2014-2026. Omnisient (Pty) Ltd. All rights reserved. - www.omnisient.com - Legal Policies

```
%omni_hub {'select current_user'}
cell first select current_user

first ['select', 'current_user\n']
second ['select', 'current_user']
cell second select current_user

Namespace(line=['select', 'current_user'], close=None, creator=None, section=None, persist=False,
Done.

2a528113-5b32-4637-995e-e876e13d49c0_f914d679-536a-4ec8-a23a-08d_Omnisient_OMNISIENT_PRJ_00114
```

Jupyter notebook output showing the current_user query result as a long alphanumeric string ending in the project name.

The username is the long alphanumeric string ending in the project name.

> Tips for managing memory: For large datasets, do as much data preparation, cleaning, joining, and sampling in SQL as possible before loading into the notebook - Python compute may run out of memory on very large datasets. Load only the data needed - a filtered query over a full table pull. Clear unused variables with Python's del, or restart the kernel to free memory between tasks.

Choosing a Workspace Size for a Query

Query performance and processing time depend on the shape of the table being pulled, and the notebook's workspace size needs to match it.

- A wide table (many columns, e.g. 500 columns across 3 million rows) is compute- and memory-intensive to process column-by-column, and typically needs the Extra Large workspace size.
- A long table (few columns, e.g. 10 columns across 50 million rows) processes differently - the Large workspace size is typically sufficient.

See Workspace size options above for the full CPU/memory breakdown of each workspace size.

> Pulling more data than a query actually needs increases the risk of the query timing out, independent of workspace size. Scope a query to the columns and rows actually needed rather than sizing up to compensate.

Optimizing Memory for Large Tables

omni_hub returns a ResultSet object. Converting it directly to a DataFrame holds the data twice in memory - once as the ResultSet, once as the DataFrame - which can cause out-of-memory errors on large tables.

Export the ResultSet to CSV, delete it, then load the CSV into pandas instead. This holds only one copy at a time:

1. Run the query with omni_hub.
2. Save the result to CSV with .csv("path").
3. Delete the original object with del.

© 2014-2026. Omnisient (Pty) Ltd. All rights reserved. - www.omnisient.com - Legal Policies

4. Load the CSV into pandas with pd.read_csv.

```
# 1) Run query via omni_hub
sql_query = "select top 10 * from [project_code].[table_name]"
rs = %omni_hub {sql_query}

# 2) Save to CSV
rs.csv("my_table.csv")

# 3) Delete the ResultSet to free memory
del rs

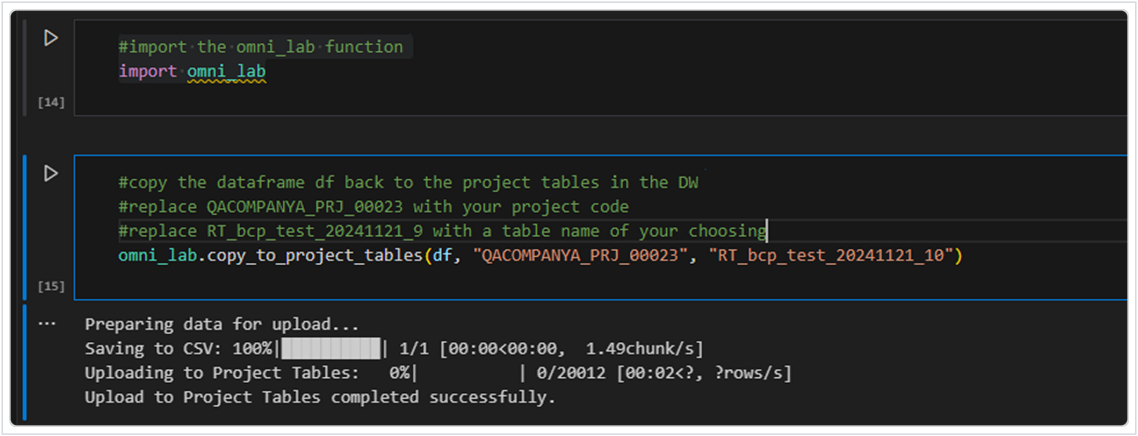
# 4) Read the CSV into pandas
import pandas as pd
df = pd.read_csv("my_table.csv", low_memory=False)
```

Writing a DataFrame Back to a Project

To write the results of an analysis back to the project tables, use omni_lab.

1. Import the package: import omni_lab
2. Copy the DataFrame back to the project: omni_lab.copy_to_project_tables(df, "PROJECT_CODE", "table_name")
3. Replace PROJECT_CODE with your project code, for example QACOMPANYA_PRJ_00023, and table_name with a name of your choosing. Table names can't contain spaces.

The write runs asynchronously.



Jupyter notebook output confirming a DataFrame write request was submitted via omni_lab.

Libraries

Import a library using standard Python syntax:

```
import pandas as pd
import numpy as np
```

To view all libraries installed on the platform, run:

```
!pip list
```

This returns an alphabetized list of all available libraries.

© 2014-2026. Omnisient (Pty) Ltd. All rights reserved. - www.omnisient.com - Legal Policies

```
[1]: # Check the list of installed packages

     !pip list

     Package                           Version
     --------------------------------- ------------
     absl-py                           2.1.0
     alembic                           1.12.0
     annotated-types                   0.7.0
     anyio                             4.0.0
     argon2-cffi                       23.1.0
     argon2-cffi-bindings              21.2.0
```

For the full list of packages and libraries available in Omnisient's Jupyter environment, see the Available Packages & Libraries for Jupyter Notebook article.

Virtual Environments

A virtual environment (venv) is a persistent, isolated Python environment within a Jupyter project, separate from the project's shared Python installation. Packages installed inside a venv persist across sessions in the project's storage, avoiding the session-only installs and version drift that come with installing packages directly in the shared environment.

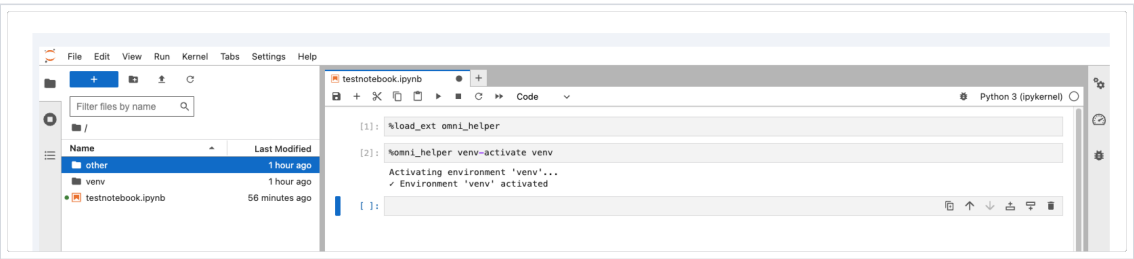
A venv is useful whenever a specific analysis or model needs a particular library version - for example, testing a newer release of a package without changing the version every other notebook in the project relies on. It also makes an analysis reproducible: recreating the same venv gives the same package versions, rather than whatever happens to be installed in the shared environment at the time.

Jupyter Notebook creates and activates project-specific venvs through the omni_helper extension.

Creating and Activating a Venv

Run these two cells at the top of the notebook, every session:

1. Load the omni_helper extension: %load_ext omni_helper
2. Create and activate the venv: %omni_helper venv-activate <env_name>. Replace <env_name> with any name you choose. The first time, this creates the venv and activates it; every time after, it just activates it.



© 2014-2026. Omnisient (Pty) Ltd. All rights reserved. - www.omnisient.com - Legal Policies

Initial creation takes 15 to 30 minutes. The venv itself persists in the project's storage, but a new session still needs to run both cells (%load_ext omni_helper and %omni_helper venv-activate venv) to reactivate it - reactivating an existing venv takes seconds.

Use the same venv name across every notebook in the project, so each one activates the same persisted venv and the packages already installed in it.

Installing Packages Inside a Venv

Once a venv is active, install a package the same way as outside one:

```
!pip install <package_name>
```

Packages install from Omnisient's vetted package library. A package installed inside an active venv persists across sessions in the project's storage - install it once, and it's available every time that venv is reactivated, without installing it again.

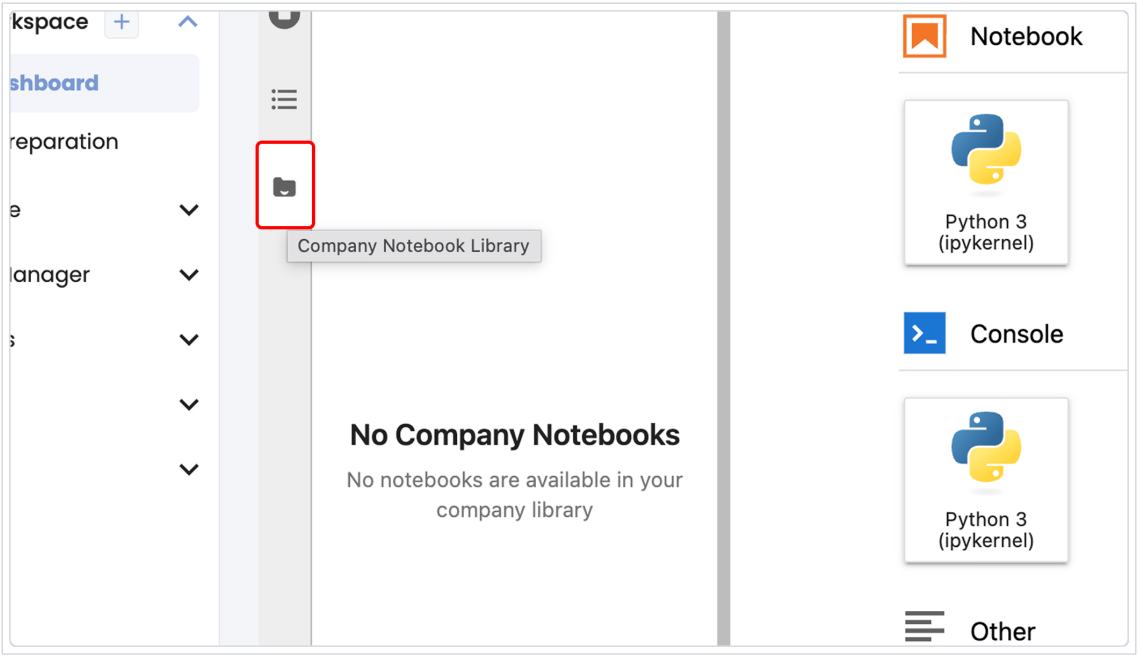
See Using Jupyter Notebook for Packages/Libraries Management for how package installs behave outside a venv.

If a package isn't part of the vetted library, it isn't available to install this way. Contact support@omnisient.com to request it be added or updated.

Company Notebook Library

The Company Notebook Library allows analysts to share approved notebooks across your organization. Once a notebook has been reviewed and approved by the Omnisient team, it becomes available to all users within the company and can be copied into any project.

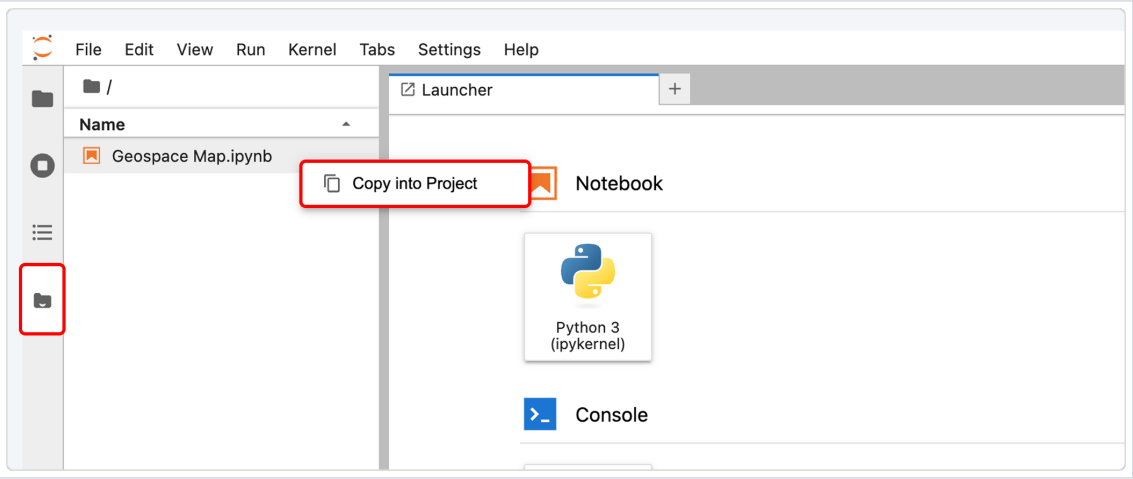
Accessing the library



© 2014-2026. Omnisient (Pty) Ltd. All rights reserved. - www.omnisient.com - Legal Policies

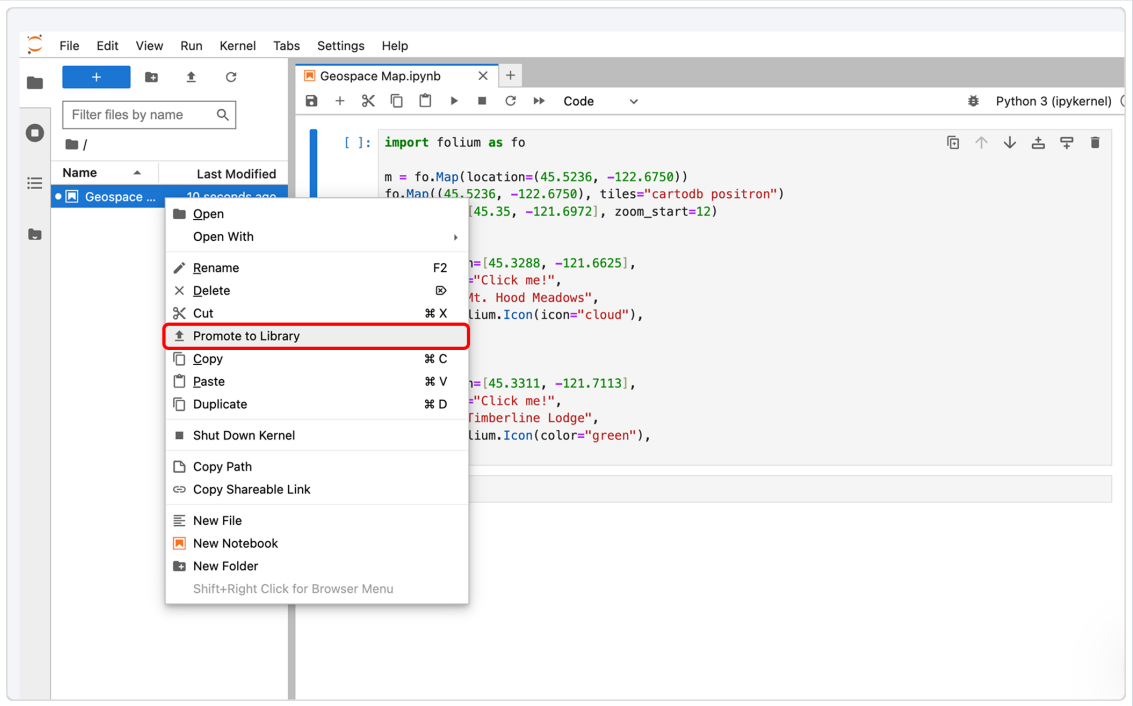
The Notebook Library panel is available in the left sidebar within Jupyter. It displays all notebooks your organization has had approved.

To use a notebook, right-click it and select Copy into Project - a copy is added to your current project's file browser.



Promoting a notebook to the library

Any analyst can submit a notebook for inclusion in the Company Notebook Library.



Clear your notebook outputs before submitting. Notebooks submitted with outputs will be declined. Before promoting, clear all outputs - go to Edit > Clear All Outputs, or Kernel > Restart Kernel and Clear All Outputs.

To submit a notebook:

1. In the file browser, right-click the notebook you want to promote.

© 2014-2026. Omnisient (Pty) Ltd. All rights reserved. - www.omnisient.com - Legal Policies

2. Select Promote to Library.
3. A confirmation modal appears - review and confirm your submission.
4. Your submission has been logged, and the Omnisient team has been notified to begin the review process.

What happens next

Once submitted, your notebook enters a review queue. The Omnisient team will review it and communicate the outcome by email.

- Approved - the notebook is added to your organization's library and becomes visible to all users in the Notebook Library panel.
- Declined - you will receive an email with the reason for the decline. Review the feedback, make the necessary changes, and resubmit.

Sharing a notebook with a project member

To share directly with a specific project member, use omni_helper.

Permissions required: Active participant in the project. Prerequisites: Jupyter Notebook open within your project.

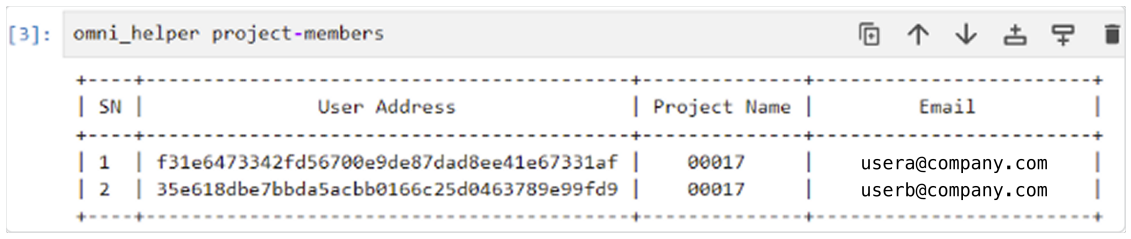
Step 1: Load the extension

```
%load_ext omni_helper
```

Step 2: Get the recipient's user address

Run omni_helper project-members to return a list of project members and their user addresses. Copy the User Address of the intended recipient.

> User addresses are project-specific - the same user has a different address in each project they belong to.



omni_helper project-members

Step 3: Send the file

```
omni_helper send-files --paths [filename.ipynb] --receiver [user_address]
```

> Filenames cannot contain spaces. Rename the file before transferring if needed.



© 2014-2026. Omnisient (Pty) Ltd. All rights reserved. - www.omnisient.com - Legal Policies

Step 4: Check transfer status

Run omni_helper transfer-history and match the output against the REQUEST_ID returned when you submitted.

Recipient: The file appears in a shared folder within their Jupyter workspace inside a dated subfolder. There is no automatic notification - they will need to check manually.

Issue	Likely cause	Fix
Recipient not listed in project-members	Not an active participant in this project	Confirm they have been added to the project
Transfer not completing	Filename contains spaces	Rename and retry
Wrong user address	Address copied from a different project	Re-run omni_helper project-members in this project

© 2014-2026. Omnisient (Pty) Ltd. All rights reserved. - www.omnisient.com - Legal Policies